Penetration Test Report – SQL Injection Authentication Bypass

Target Application: https://demo.owasp-juice.shop/#/
Test Date: 12/09/2025
Tester: Amaan Tamboli

1. Vulnerability Title

SQL Injection – Authentication Bypass

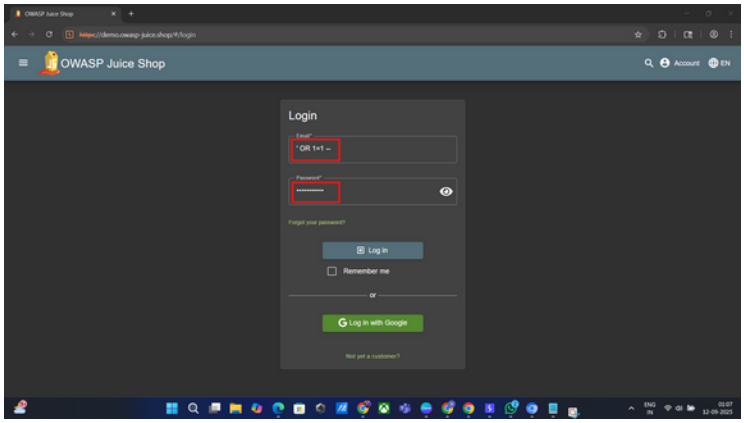
2. Vulnerability Description

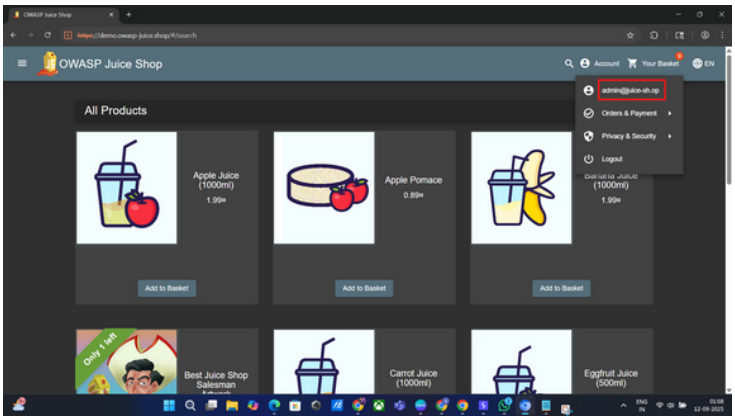
The login functionality of the OWASP Juice Shop is vulnerable to SQL Injection. By injecting SQL payloads into the email and password fields, attackers can manipulate backend queries and bypass authentication without valid credentials.
This flaw occurs because user-supplied input is not properly sanitized before being included in SQL statements.

3. Proof of Concept

Injected Query (simplified): SELECT * FROM users WHERE email = '' OR 1=1 --' AND password = '' OR 1=1 --';

- The condition OR 1=1 always evaluates to TRUE.
- The -- sequence comments out the rest of the SQL query.
- As a result, authentication checks are bypassed.





4. Steps to Reproduce

- Navigate to the login page: https://demo.owasp-juice.shop/#/login
- Enter the following credentials:
  - Email: ' OR 1=1 --
  - Password: ' OR 1=1 --
- Submit the form.
- Result: The login succeeds without valid credentials, granting unauthorized access.

5. Impact

- Confidentiality: Unauthorized attackers gain access to user accounts.
- Integrity: Attackers can impersonate users, including administrators, and modify data.
- Availability: Exploitation can lead to further compromise of the entire system.

6. Severity Rating

- CVSS v3.1 Base Score: 9.8 (Critical)
- Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

7. Recommendations

- Use parameterized queries (prepared statements) to prevent SQL injection.
- Implement server-side input validation and sanitization.
- Enforce least privilege access for database accounts.
- Conduct regular penetration testing and use web application firewalls (WAFs) to detect suspicious queries.

8. References

- OWASP SQL Injection
- OWASP Juice Shop

Penetration Test Report – Insecure Transmission of Credentials

Target Application: https://demo.owasp-juice.shop/#/
Test Date: 12/09/2025
Tester: Amaan Tamboli

1. Vulnerability Title

Insecure Transmission of Credentials – Sensitive Data Exposure

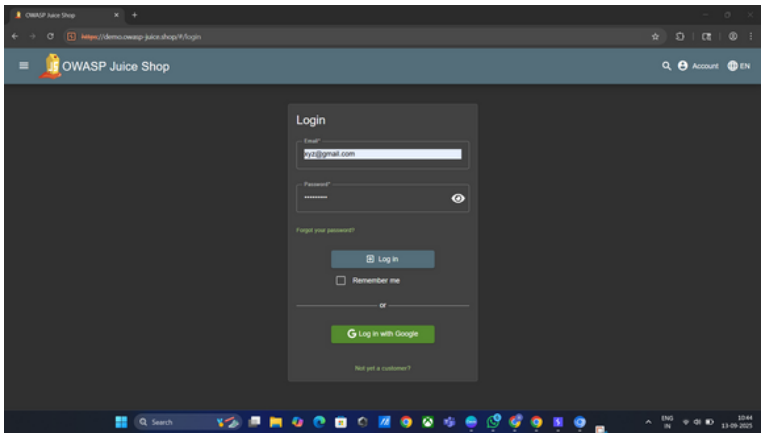
2. Vulnerability Description

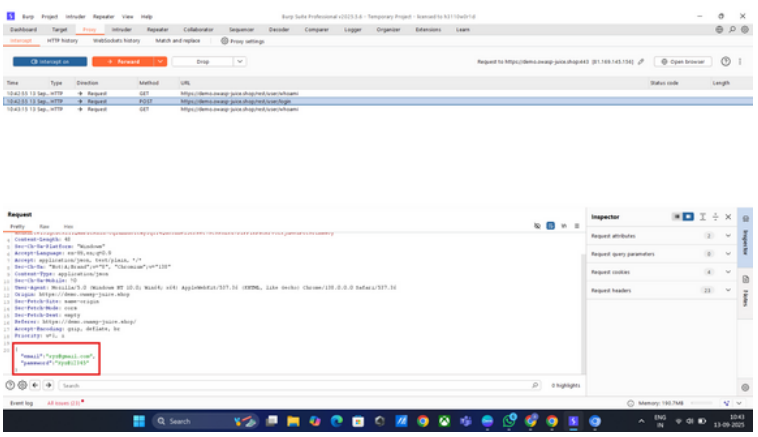
During testing, it was observed that user credentials (email and password) are transmitted in cleartext within the login request. When intercepting the request using a proxy tool (e.g., BurpSuite), both the username (email) and password are directly visible.
This issue falls under Sensitive Data Exposure (OWASP Top 10) because authentication data is being transmitted in a manner that can be intercepted and logged, putting users at risk if traffic is captured or logs are exposed.

3. Proof of Concept

- Credentials were entered into the login form:
  - Email: xyz@gmail.com
  - Password: Xyz@12345
- On interception, the request contained these credentials in a readable format (see attached screenshot in evidence section).

This demonstrates that user authentication data is not adequately protected during transmission.





4. Steps to Reproduce

- Navigate to the login page: https://demo.owasp-juice.shop/#/login
- Enter valid credentials and submit the login form.
- Intercept the request using a web proxy tool (e.g., BurpSuite).
- Observe that the email and password fields are visible in plaintext within the intercepted request.

5. Impact

- Confidentiality: User credentials are at risk of being exposed via interception, proxy logs, browser history, or monitoring systems.
- Integrity: Attackers with access to intercepted traffic can hijack accounts or escalate privileges.
- Compliance Risk: Storing/transmitting passwords insecurely may violate GDPR, PCI DSS, or other regulatory requirements.

6. Severity Rating

- CVSS v3.1 Base Score: 7.5 (High)
- Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

7. Recommendations

- Use HTTPS/TLS for all communication to ensure credentials are encrypted in transit.
- Avoid including sensitive information in URLs or query parameters. Instead, transmit them in a POST body over TLS.
- Implement secure session management and ensure credentials are never logged in server logs.
- Consider adding multi-factor authentication (MFA) for additional protection.

8. References

- OWASP: Sensitive Data Exposure
- OWASP Top 10: A02 – Cryptographic Failures
- OWASP Juice Shop

Penetration Test Report – SQL Injection via Burp Suite Intruder (Authentication Bypass)

Target Application: https://demo.owasp-juice.shop/#/
Test Date: 12/09/2025
Tester: Amaan Tamboli

1. Vulnerability Title

SQL Injection – Authentication Bypass (Automated Payload Discovery)

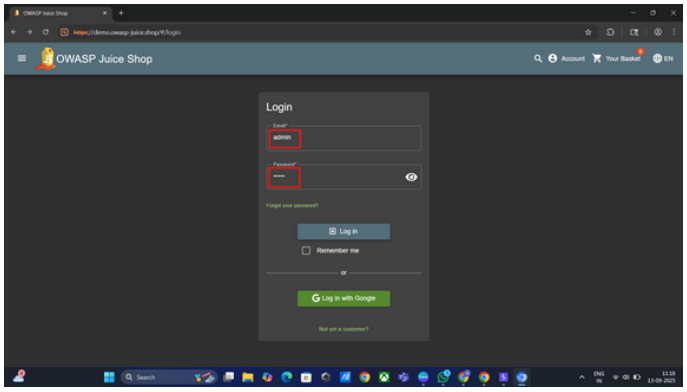
2. Vulnerability Description

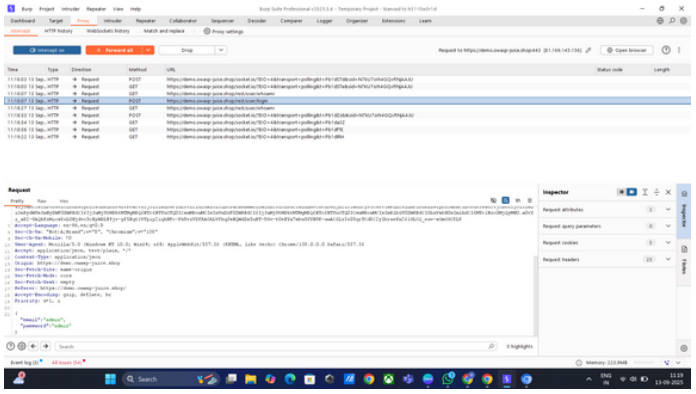
The login endpoint of the OWASP Juice Shop is vulnerable to SQL Injection. By manipulating the uid (username) and pw (password) parameters, attackers can inject malicious SQL payloads that alter backend queries.
Using Burp Suite Intruder with a cluster bomb attack, multiple payload combinations successfully bypassed authentication and granted access to the admin account.
This vulnerability exists because the application fails to properly sanitize and parameterize user input before incorporating it into SQL queries.

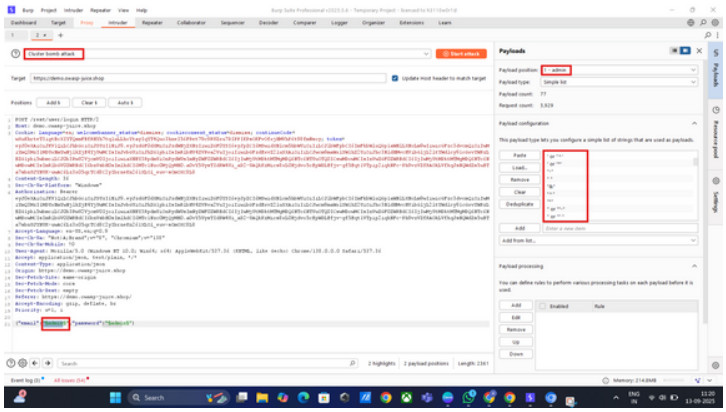
3. Proof of Concept

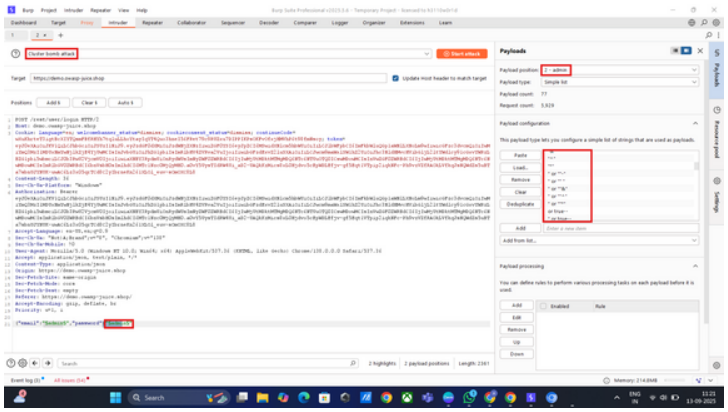
- Targeted Request (captured in Burp):
  POST /rest/user/login HTTP/2
  Host: demo.owasp-juice.shop
  Content-Type: application/json

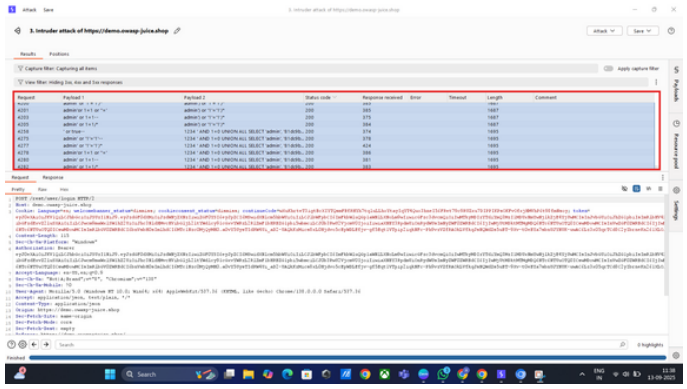
  {"uid":"admin","pw":"admin"}
- Attack Method:
  - Sent request to Burp Suite Intruder.
  - Changed parameters to test: uid=$admin$&pw=$admin$
  - Configured Cluster Bomb attack with payload lists for both username and password containing SQL injection strings.
  - Disabled URL encoding.
  - Launched attack.
- Successful Payload Examples (leading to admin login):
  - uid=admin' or '1'='1'-- + pw='-'
  - uid=admin' or 1=1/* + pw=' '
  - uid=' or true-- + pw=' or true--
  - uid=admin'or 1=1 or ''= + pw='*'
  - uid=admin" or "1"="1"-- + pw='1234 ' AND 1=0 UNION ALL SELECT 'admin','81dc9bdb52d04dc20036dbd8313ed055











Observation:

- Multiple payloads returned HTTP 200 OK with larger response lengths (e.g., 1687–1703 bytes).
- Successful attempts granted unauthorized access to the admin account.

4. Steps to Reproduce

- Navigate to the login page: https://demo.owasp-juice.shop/#/login
- Intercept the login request using Burp Suite.
- Modify the request to: {"uid":"$admin$","pw":"$admin$"}
- Send to Intruder → Configure Cluster Bomb Attack.
  - Payload Set 1 (username): Insert SQL injection strings (' or true--, admin' or '1'='1'--, admin' or 1=1/*, etc.).
  - Payload Set 2 (password): Insert same SQL injection strings.
  - Disable URL encoding.
- Launch attack and monitor responses.
- Successful payloads produced larger response bodies and authenticated as admin.

5. Impact

- Confidentiality: Attackers can gain access to sensitive user/admin accounts, exposing personal information, purchase history, and stored data.
- Integrity: Admin access allows manipulation of product listings, pricing, and database records. Attackers could plant malicious content or manipulate orders.
- Availability: Exploitation may lead to denial of service by deleting or corrupting key data.
- Privilege Escalation: Attackers gain full admin-level access without credentials, enabling system-wide compromise.

6. Severity Rating

- CVSS v3.1 Base Score: 9.8 (Critical)
- Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

7. Recommendations

- Implement parameterized queries (prepared statements) to ensure user input cannot alter SQL queries.
- Use stored procedures instead of inline dynamic SQL where possible.
- Apply server-side input validation and sanitization.
- Employ least privilege on database accounts to limit impact.
- Monitor logs for suspicious authentication attempts and implement rate limiting.
- Use Web Application Firewalls (WAFs) to block common SQL injection payloads.
- Conduct secure code reviews and regular penetration testing to identify injection points early.

8. References

- OWASP SQL Injection
- OWASP Juice Shop
- OWASP Top 10 – A03 Injection

Penetration Test Report – Cross-Site Scripting (XSS) in Search Functionality

Target Application: https://demo.owasp-juice.shop/#/
Test Date: 13/09/2025
Tester: Amaan Tamboli

1. Vulnerability Title

Cross-Site Scripting (XSS) – Search Input Field

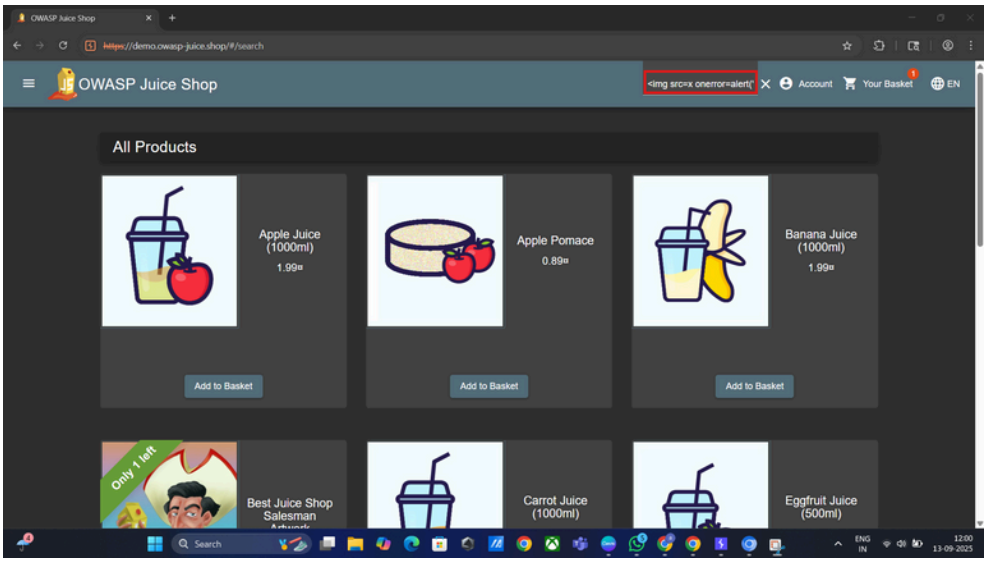
2. Vulnerability Description

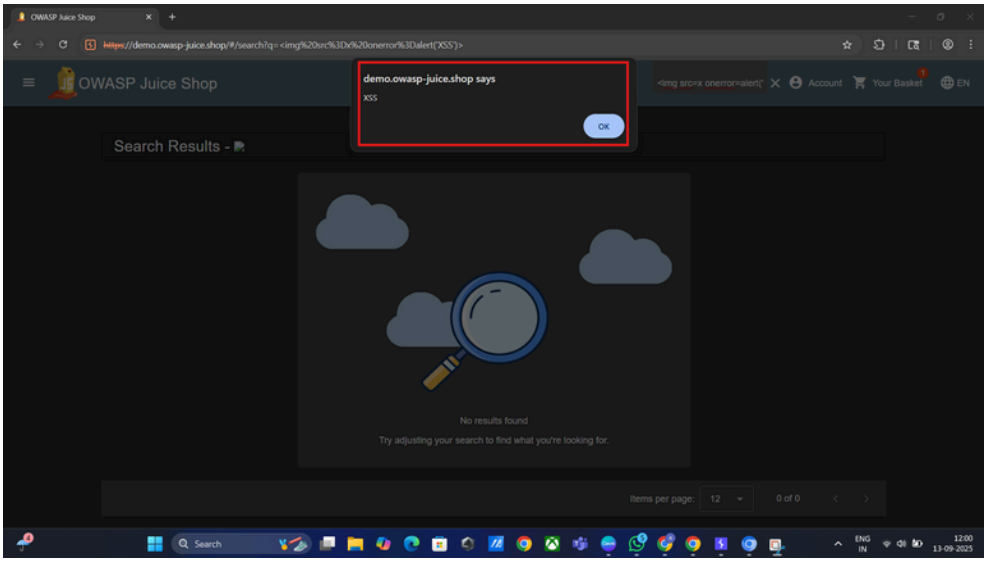
The search functionality of the application is vulnerable to Cross-Site Scripting (XSS). By injecting malicious JavaScript code into the search input field, an attacker can execute arbitrary scripts in the context of the victim's browser.
This occurs because the application fails to properly sanitize or encode user input before reflecting it back to the page.

3. Proof of Concept (PoC)

Payload Used: <img src=x onerror=alert('XSS')>
Steps:

- Navigate to the application and log in.
- Locate the search input field.
- Enter the payload above into the search field and submit.
- Observe that an alert box pops up with XSS, confirming that arbitrary JavaScript execution is possible.





4. Impact

- Confidentiality: Attackers could steal session cookies, tokens, or sensitive information from victims.
- Integrity: Malicious scripts could alter the content of the web page, misleading users.
- Availability: Attackers could inject scripts causing denial of service (e.g., infinite alerts, page crashes).
- Exploitation: If combined with social engineering, attackers can hijack user sessions or deliver malware.

5. Severity Rating

- CVSS v3.1 Base Score: 7.4 (High)
- Vector: AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:L

6. Recommendations

- Implement proper output encoding (e.g., HTML entity encoding) for user-supplied input.
- Apply server-side input validation and sanitization to block malicious payloads.
- Use Content Security Policy (CSP) headers to restrict execution of inline scripts.
- Employ HttpOnly cookies to protect session data from being accessed via JavaScript.
- Conduct regular security testing for input fields that handle user-generated content.

7. References

- OWASP XSS
- OWASP Top 10 – A03: Injection
- Content Security Policy (CSP)

Penetration Test Report – Insecure Direct Object Reference (IDOR) in Basket Functionality

Target Application: https://demo.owasp-juice.shop/#/
Test Date: 13/09/2025
Tester: Amaan Tamboli

1. Vulnerability Title

Insecure Direct Object Reference (IDOR) – Basket Functionality

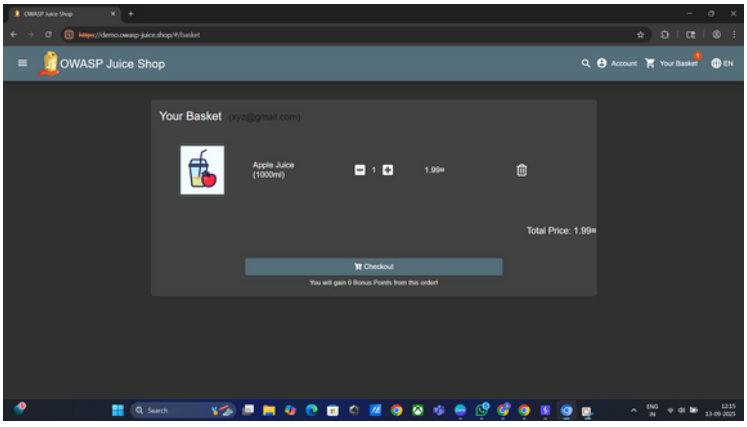
2. Vulnerability Description

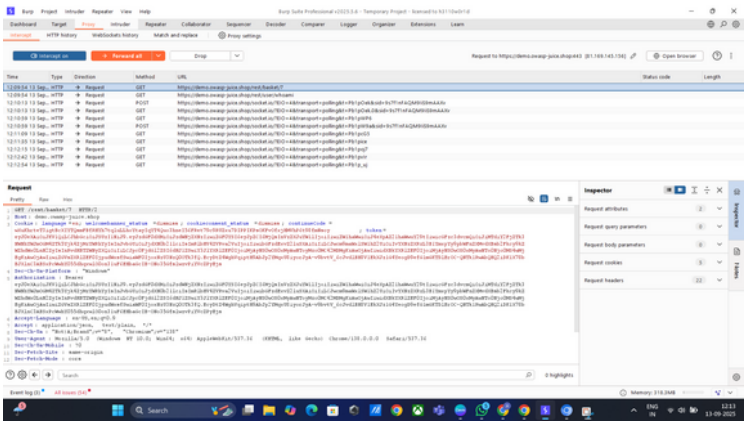
The application exposes Insecure Direct Object Reference (IDOR) in the basket API endpoint. When a logged-in user retrieves their basket contents via: GET /rest/basket/{basketId}
The server does not validate whether the requested basketId belongs to the authenticated user. This allows an attacker to manipulate the basketId value and gain unauthorized access to other users' basket data.

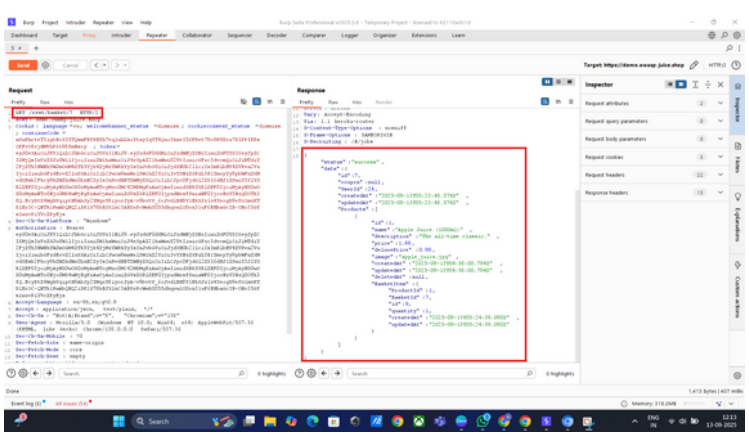
3. Proof of Concept (PoC)

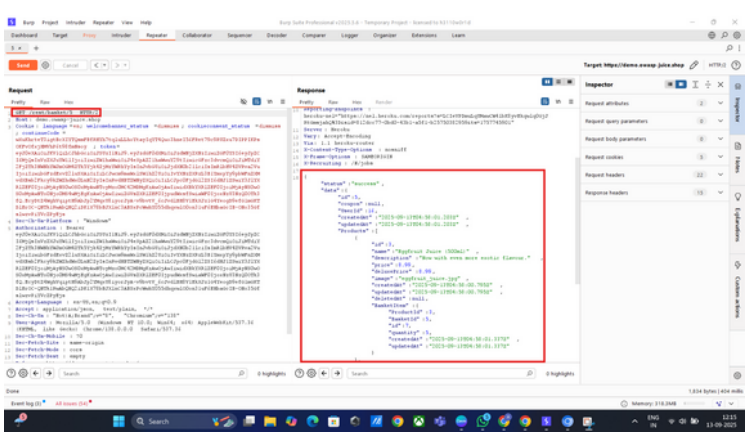
Steps to Reproduce:

- Log in with valid credentials:
  - Email: xyz@gmail.com
  - Password: Xyz@12345
  - URL: https://demo.owasp-juice.shop/#/login
- Add a product to your basket:
  - Navigate to: https://demo.owasp-juice.shop/#/basket
- Intercept the request in Burp Suite: GET /rest/basket/7 HTTP/2
- Change the basketId value from 7 to another ID, for example: GET /rest/basket/5 HTTP/2
- Observe: You can access another user's basket contents, confirming the IDOR vulnerability.









4. Impact

- Confidentiality: Attackers can view other users' basket items, potentially exposing sensitive order or purchase data.
- Integrity: Attackers could manipulate basket contents, adding or removing products from another user's basket.
- Business Risk: May lead to fraudulent purchases, privacy violations, and reputational damage.

5. Severity Rating

- CVSS v3.1 Base Score: 8.2 (High)
- Vector: AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L

6. Recommendations

- Implement object-level access control: Ensure that the requested basketId belongs to the authenticated user.
- Enforce server-side authorization checks for all sensitive endpoints.
- Use session-bound identifiers rather than sequential or guessable IDs.
- Perform regular access control testing during security assessments.

7. References

- OWASP Top 10 – A01: Broken Access Control
- CWE-639: Authorization Bypass Through User-Controlled Key

Penetration Test Report – Authentication Bypass via SQL Injection

Target Application: https://demo.owasp-juice.shop/ (Login API: /rest/user/login)
Test Date: 29/09/2025
Tester: Amaan Tamboli

1.Vulnerability Title

Authentication Bypass via SQL Injection in Login Endpoint

2.Vulnerability Description

The login endpoint fails to properly sanitize user-supplied input in the email field. By injecting SQL metacharacters (for example, a single quote followed by a comment) into the email parameter, an attacker can alter the backend query logic and cause the application to return a successful authentication response (token) without valid credentials. This indicates the authentication query is built using unsafely concatenated input and is vulnerable to SQL injection.

3.Proof of Concept (PoC)

Valid Login for Comparison:

- Email: bender@juice-sh.op
- Password: test

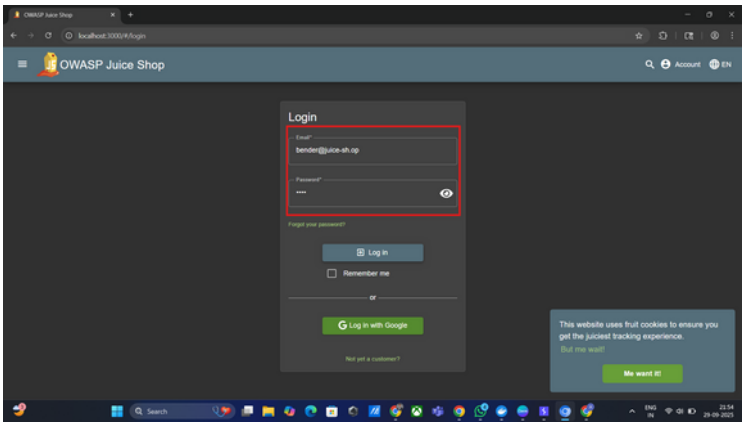
Malicious Payloads:

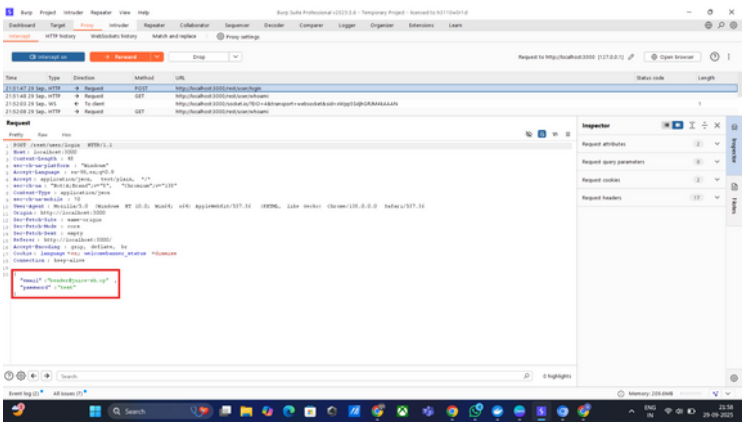
- Email: bender@juice-sh.op'--
- Password: test (unchanged)

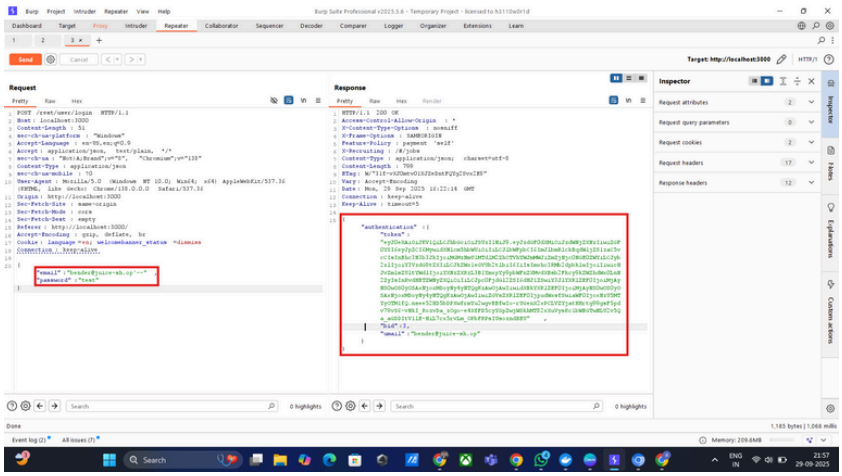
Observed Response (truncated):
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"authentication":{"token":"<JWT token>","bid":3,"umail":"bender@juice-sh.op"}}

Observation: Submitting the altered email parameter returned HTTP 200 with a valid authentication token and user context (umail/bid) for the account, demonstrating authentication was bypassed via injection.







4.Steps to Reproduce

- Open the application login page: https://demo.owasp-juice.shop/#/login.
- Intercept the login request with a proxy (e.g., Burp Suite).
- Submit valid credentials once to capture a baseline request ({"email":"bender@juice-sh.op","password":"test"}).
- Send the captured request to Repeater (or edit inline in the proxy).
- Modify the email parameter to: bender@juice-sh.op'-- and leave password as test.
- Forward the modified request to the server.
- Observe the server returns HTTP 200 OK with JSON containing an authentication token and umail = bender@juice-sh.op.
- Use the token in subsequent requests (Authorization header) to confirm authenticated access as the targeted user.

5.Impact

- Confidentiality: Attackers can obtain valid authentication tokens for user accounts and access private data (orders, profile info).
- Integrity: With account access, attackers can modify user data, place orders, or post content on behalf of users.
- Availability: Mass exploitation could lead to abuse, fraudulent transactions, and operational disruption.
- Privilege escalation: If admin or privileged accounts are susceptible, the entire application could be compromised.
- Business risk: Data breach, fraud, reputational damage, and regulatory exposure.

6.Severity Rating

- CVSS v3.1 Base Score: 9.8 (Critical)
- Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

7.Recommendation

- Immediate (code): Replace any string-concatenated SQL with parameterized queries / prepared statements for all database interactions, especially authentication.
- Validation: Add input validation and normalization, but do not rely on validation alone to prevent SQLi.
- Least privilege: Ensure the DB account used by the application has minimal privileges.
- WAF (temporary): Deploy a Web Application Firewall with SQLi protections as a short-term mitigation while code is fixed.
- Logging & detection: Log and alert on anomalous authentication patterns and token issuance.
- Testing: Run SAST/DAST and targeted SQLi fuzzing against all query entry points; add regression tests to CI.
- Rotate tokens / credentials: If exploitation is suspected, revoke issued tokens and force password resets as appropriate.

8.References

OWASP: SQL Injection — https://owasp.org/www-community/attacks/SQL_Injection
OWASP Top 10 — Injection (A03 2021) — https://owasp.org/Top10/
OWASP Juice Shop (training target) — https://owasp.org/www-project-juice-shop/.

Penetration Test Report – Insecure Direct Access & Directory Listing

Target Application: https://demo.owasp-juice.shop/
Test Date: 29/09/2025
Tester: Amaan Tamboli

1.Vulnerability Title

Insecure Direct Access / Directory Listing and Arbitrary File Exposure (FTP directory accessible via web)

2.Vulnerability Description

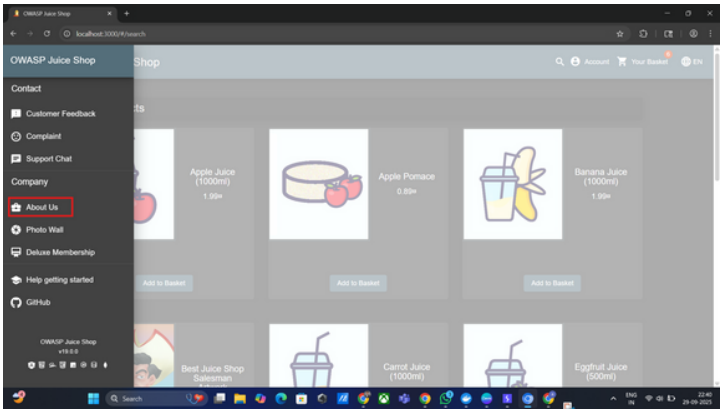
The application exposes an /ftp/ path that returns a browsable directory listing and allows access to individual files (e.g., /ftp/legal.md). An About-page link navigates to /ftp/legal.md; by modifying the URL the tester accessed https://demo.owasp-juice.shop/ftp/ and saw a full directory listing. The listing contains potentially sensitive files (e.g., incident-support.kdbx, announcement_encrypted.md, encrypt.pyc, backup files). This indicates directory indexing and insufficient access controls for files that should be private. The application also allows editing of legal.md via the UI, implying write capability may exist for those files.

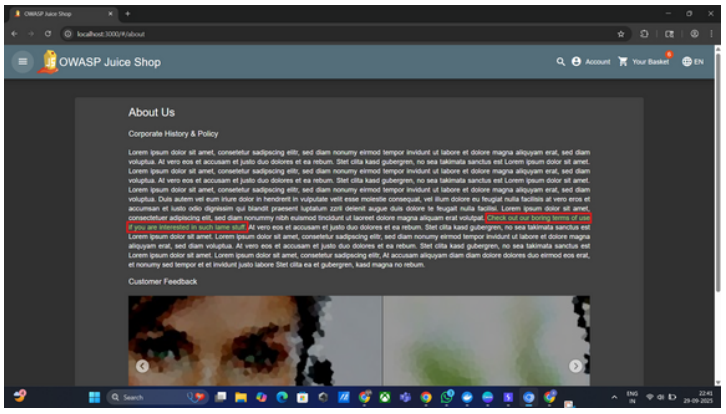
3.Proof of Concept (PoC)

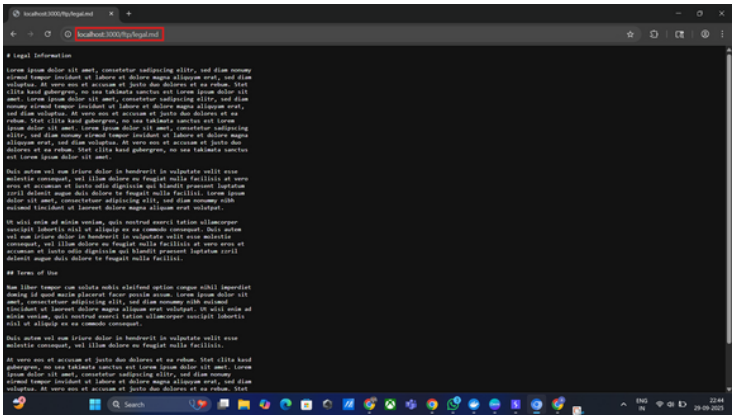
Normal behavior: About page links to terms file.

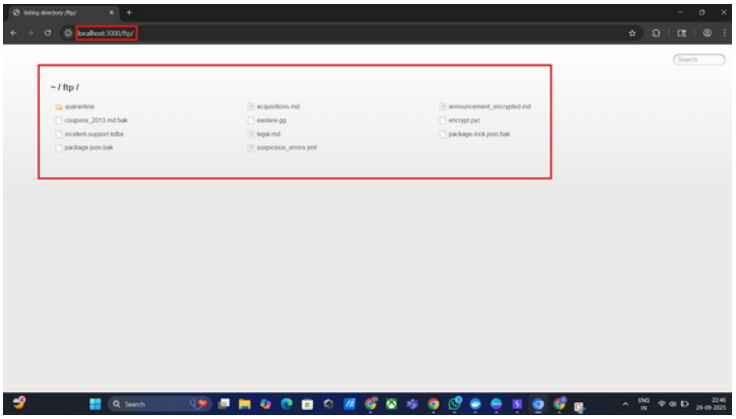
PoC steps and observations:

- Logged in and opened the About page; clicked the terms link which navigates to /ftp/legal.md.
- Edited/inspected the URL and navigated to /ftp/.
- Received an HTML directory listing showing multiple files and folders (screenshot captured).
- Files visible include incident-support.kdbx, acquisitions.md, announcement_encrypted.md, encrypt.pyc, suspicious_errors.yml, package.json.bak etc.
- Confirmed ability to view file contents; observed that legal.md could be edited/saved via application flow, indicating write/update capability.









4.Steps to Reproduce

- Authenticate to the application (if required).
- Navigate to the About / “terms” link that points to /ftp/legal.md.
- In the browser address bar, change the path to /ftp/ (e.g., https://demo.owasp-juice.shop/ftp/) and press Enter.
- Observe the directory listing containing multiple files.
- Click any filename to view its contents.
- (Optional) Attempt to edit legal.md via the site UI and submit changes; observe acceptance.
- Capture screenshots of the directory listing and file contents.
- Record the HTTP requests/responses (proxy) showing access to /ftp/ and files.

5.Impact

- Confidentiality: High — exposed files may contain secrets, backups, credentials (e.g., KeePass DB, encrypted announcements, error logs).
- Integrity: High — if write/edit is possible, an attacker could tamper with files, inject malicious content, or place phishing/malicious pages.
- Availability: Medium — attackers could delete or corrupt files, impacting operations.
- Business risk: Critical — potential data breach, credential compromise, regulatory impact and reputational damage.
- Exploitability: Trivial for read access (just browse), severe if write access is available.

6.Severity Rating

- CVSS v3.1 Base Score: 9.0 (Critical) — suggested
- Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

7.Recommendation

- Use parameterized queries / prepared statements instead of string concatenation.
- Implement server-side input validation for email/password fields.
- Apply least privilege to the database user account.
- Deploy a WAF to block SQLi attempts until code is fixed.
- Enable logging & monitoring for suspicious login activity.

8.References

OWASP Top 10 — A01: Broken Access Control — https://owasp.org/Top10/
OWASP: Insecure File Handling & Uploads — https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
Apache autoindex / Nginx autoindex docs (disable directory listing)

Penetration Test Report – Valid Account Authentication (Account Access)

Target Application: https://demo.owasp-juice.shop/ (Login: /rest/user/login)
Test Date: 29/09/2025
Tester: Amaan Tamboli

1.Vulnerability Title

Valid Account Authentication — Confirmed access using discovered credentials

2.Vulnerability Description

The tester successfully authenticated to the application using the credential pair below. Presence of valid, potentially easily-discoverable or reused credentials (or credentials obtained during testing) represents an account access risk. If these credentials were obtained via weak provisioning, public disclosure, credential stuffing, or previous tests, they demonstrate that an attacker with the same information can gain full access to the associated user account and any functionality available to that role.

3.Proof of Concept (PoC)

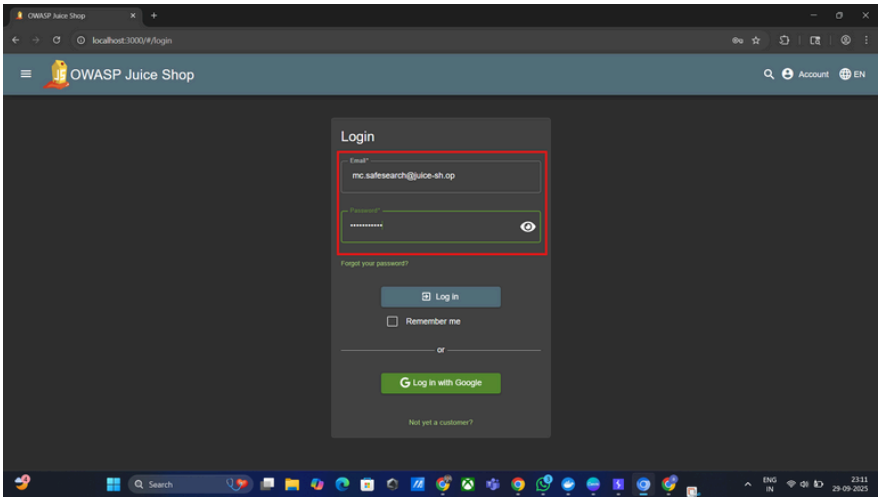
Valid Login:

- Email: mc.safesearch@juice-sh.op
- Password: Mr. N00dles

Observed Behavior: Successful authentication to the application. Server returned an authenticated session (HTTP 200 and a valid authentication token / session context), and tester was able to access user-only pages (profile, snippets, etc.). (Capture of request/response and token available in testing evidence.)

4.Steps to Reproduce

- Navigate to the login page: https://demo.owasp-juice.shop/#/login.
- Enter credentials: Email mc.safesearch@juice-sh.op and Password Mr. N00dles.
- Submit the form.
- Observe successful login and redirection to authenticated user pages.
- (Optional) Intercept the login request in a proxy (Burp) and confirm HTTP 200 response and returned authentication token in JSON.
- Use the token in an Authorization header to access protected API endpoints (confirm authenticated access).
- Capture screenshots / proxied requests as evidence.
- Log out and re-test to confirm consistent behavior.



5.Impact

- Confidentiality: An attacker with the credentials can view private user data (profile, snippets, orders) and any other resources available to this account.
- Integrity: The attacker can modify account data, create or delete content, and perform actions as the user.
- Availability: Account misuse can lead to fraudulent actions (orders, content spam) and operational disruption.
- Business risk: If the account is privileged or reused on other services, this can lead to greater compromise and reputational/regulatory impact.

6.Severity Rating

- CVSS v3.1 Base Score: 6.5 (Medium) — suggested
- Vector: AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:H/A:L
- (Rationale: ability to access an account is a clear risk. Severity depends on the account's privilege level and how the credentials were obtained. If credentials were leaked or obtained via attack (SQLi, directory listing, etc.), severity increases to High/Critical.)

7.Recommendation

- Rotate the exposed credentials immediately (force password reset for the account).
- Enforce strong password policy (minimum length, complexity, block common/reused passwords).
- Enable multi-factor authentication (MFA) for all user accounts, especially for administrative roles.
- Implement credential-attack protections: account lockout, rate limiting, login anomaly detection, and IP/geolocation monitoring.
- If credentials were discovered via prior testing (SQLi, directory listing, public repo), remediate the root cause and rotate all impacted secrets.
- Log and monitor successful/failed login attempts; alert on suspicious patterns.
- Educate users to avoid password reuse and consider adding breached-password checks (e.g., haveibeenpwned API).

8.References

- OWASP: Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP: Credential Stuffing Prevention — https://owasp.org/www-project-credential-stuffing/
- NIST SP 800-63B: Digital Identity Guidelines (password and MFA guidance).

Penetration Test Report – Arbitrary File Download / Directory Traversal (Null-Byte Bypass)

Target Application: https://demo.owasp-juice.shop/ (FTP directory)
Test Date: 29/09/2025
Tester: Amaan Tamboli

1.Vulnerability Title

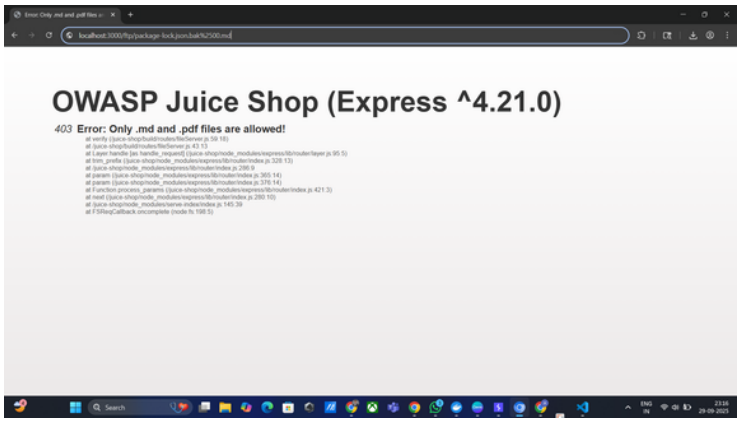
Arbitrary File Download via Path Manipulation / Null-Byte (URL encoded) Bypass

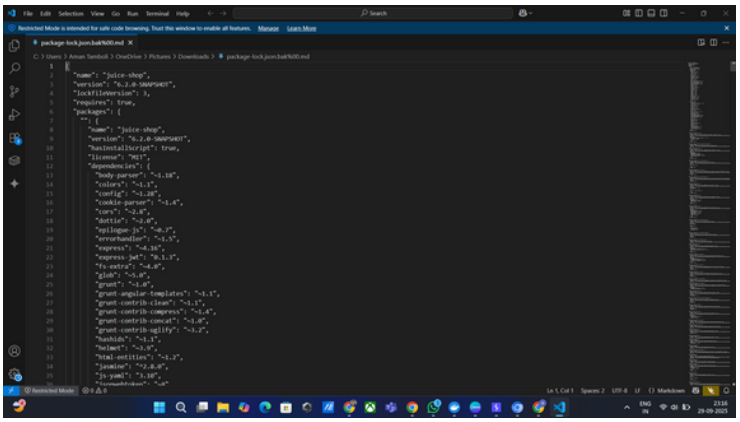
2.Vulnerability Description

The application's /ftp/ endpoint permits direct downloading of files in the web-accessible directory, including backup and sensitive artifacts. By requesting a specially-crafted path that includes a URL-encoded null byte sequence (%2500 → %00 when decoded), the server returns the contents of files that would otherwise be filtered or protected (example: /ftp/package.json.bak%2500.md causes the server to return package.json.bak). This indicates insufficient path normalization and input sanitization, and a failure to restrict access to sensitive files (backup files, config, keys) in the webroot.

3.Proof of Concept (PoC)

Valid baseline: /ftp/ directory listing is accessible and /ftp/legal.md is viewable.
PoC request used (example):
GET /ftp/package.json.bak%2500.md HTTP/1.1
Host: demo.owasp-juice.shop
Observed behavior: The request triggers a file download of the package.json.bak file (the server returns the backup file content despite the .md suffix), confirming a null-byte / path handling bypass and ability to retrieve backup/config files.





4.Steps to Reproduce

- Authenticate to the application (if required).
- Browse to the exposed directory: https://demo.owasp-juice.shop/ftp/ and note filenames (e.g., package.json.bak).
- In the browser address bar (or using curl/Burp), request the file with a URL-encoded null byte + extension suffix, for example:
- https://demo.owasp-juice.shop/ftp/package.json.bak%2500.md
- Press Enter / send the request.
- Observe that the server returns the contents of package.json.bak as a downloadable response (file served).
- Save the downloaded file and inspect contents to confirm sensitive information (e.g., config, secrets, backups).
- (Optional) Repeat with other backup/hidden filenames observed in the /ftp/ listing.
- Capture the HTTP request/response in Burp (sanitized) for evidence.

5.Impact

- Confidentiality: High — attackers can download backup/config files that may contain credentials, API keys, deployment data, or other sensitive material.
- Integrity: High — exposure of internal artifacts can enable further targeted attacks, social engineering, or tampering.
- Availability: Medium — attackers could use exposed information to disrupt services.
- Business risk: Critical — disclosure of secrets, credentials or internal configs may lead to full application compromise and regulatory/brand impact.

6.Severity Rating

- CVSS v3.1 Base Score: 9.0 (Critical) — suggested
- Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
- (Rationale: Remote attacker can obtain sensitive files without privileges, enabling credential disclosure and escalation.)

7.Recommendation

- Remove sensitive/backup files (*.bak, configs, secrets) from the webroot.
- Disable directory listing on the server.
- Implement strict allow-listing for served file types.
- Sanitize and canonicalize file paths, blocking null-byte (%00) sequences.
- Move sensitive artifacts to non-public storage.
- Rotate any exposed credentials immediately.

8.References

- OWASP: Path Traversal — https://owasp.org/www-community/attacks/Path_Traversal
- OWASP: Insecure File Handling — https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- CWE-73: External Control of File Name or Path — https://cwe.mitre.org/data/definitions/73.html

Penetration Test Report – Insecure Password Recovery (Account Takeover via Guessable Security Question)

Target Application: https://demo.owasp-juice.shop/
Test Date: 29/09/2025
Tester: Amaan Tamboli

1.Vulnerability Title

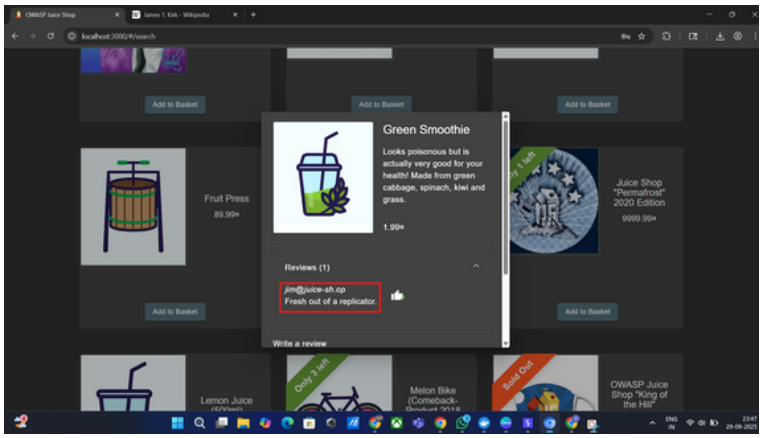
Insecure Password Recovery — Account takeover via guessable / publicly-derived security question answer

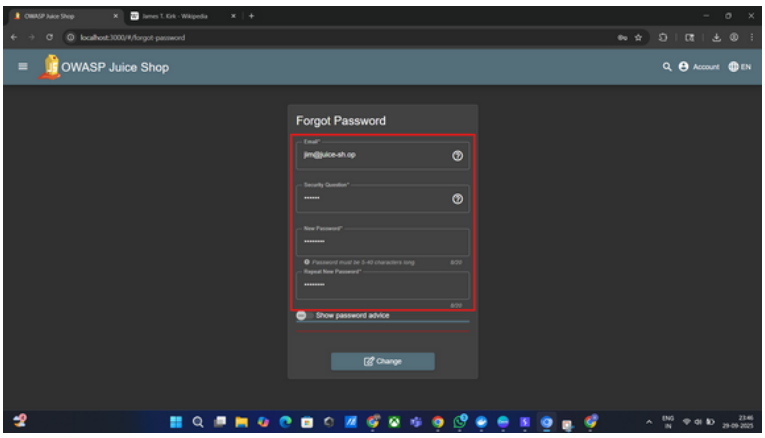
2.Vulnerability Description

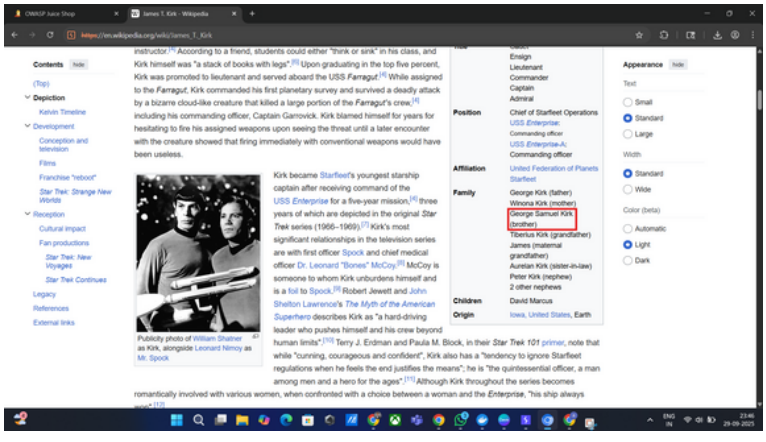
The application's password-reset / account-recovery flow relies on a knowledge-based security question whose correct answer can be derived from publicly-available information (user comments + simple web search). An attacker used information exposed in a user comment (reference to "replicator" → Star Trek → identified brother "George Samuel Kirk") to answer the target user's security question and successfully reset the account password. This allows an attacker to take full control of the victim's account without knowing the original password.

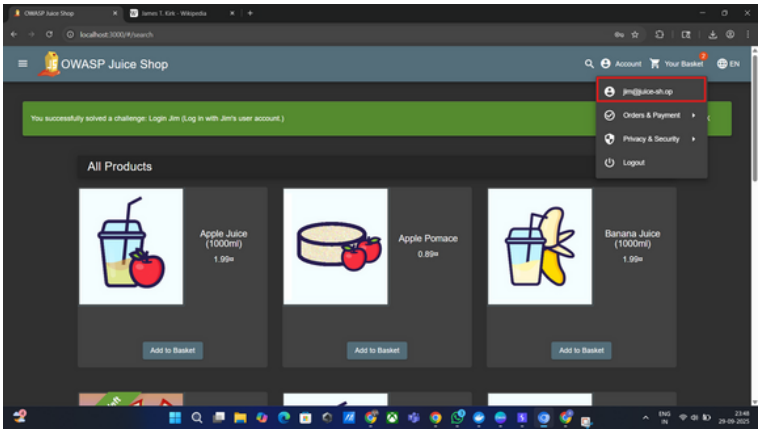
3.Proof of Concept (PoC)

Evidence chain: A comment authored by jim@juice-sh.op contained the phrase "fresh out of a replicator". A quick web search for "replicator" returned Star Trek context; further lookups tied the referenced character to George Samuel Kirk (brother).
Using the deduced name (Samuel) as the answer to the password-recovery security question for the target account, the tester was able to set a new password and then log in to the victim account successfully.
Observation: The password reset workflow accepted an unverified knowledge-based answer derived from public data and allowed immediate password replacement without additional verification (email, 2FA, one-time code).









4.Steps to Reproduce

- Identify a target account that has a recoverable security question (e.g., jim@juice-sh.op) or find a user-controlled field containing public clues (comments, profile text).
- Inspect the user's public comments/postings — note the phrase: "fresh out of a replicator."
- Use a web search (Google) for "replicator" → find Star Trek references and related character names.
- From Star Trek material, identify the family/related character name → George Samuel Kirk (brother).
- Navigate to the application's password recovery / reset flow for the target account.
- When prompted for the security question (e.g., "What is your brother's name?"), enter the derived answer: Samuel.
- Submit the recovery form, set a new password when allowed.
- Log in with the new password to confirm successful account takeover.

5.Impact

- Confidentiality: Full account access allows viewing of personal data, private messages, orders, and any sensitive information within the account.
- Integrity: Attacker can modify account details, post content, create or delete data, and perform actions as the user.
- Availability: Account abuse may lock out legitimate user, cause fraudulent orders, or otherwise disrupt service for the victim.
- Business risk: Account takeover leads to fraud, reputational damage, regulatory exposure (if PII is accessed), and possible lateral attacks if credentials are reused elsewhere.
- Exploitability: High — requires only public reconnaissance and a single password reset operation.

6.Severity Rating

- CVSS v3.1 Base Score: 8.8 (High) — suggested
- Vector: AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:L
- (Rationale: Remote attacker can take over accounts using public information; no privilege required and no user interaction beyond the attacker's actions.)

7.Recommendation

- Remove or replace knowledge-based questions with stronger recovery methods (one-time email links, SMS/Authenticator-based OTP).
- Require verified out-of-band confirmation for password resets (send expiring token to the account email; require click-through).
- Enforce rate-limiting & anomaly detection on recovery attempts (lock or CAPTCHA after several failed attempts).
- Add multi-factor authentication (MFA) and require step-up auth for sensitive changes.
- Avoid storing or displaying public clues that can be used to answer recovery questions
- Log and alert on sensitive account recovery events and require re-authentication for high-risk actions.
- Educate users to avoid using easily-guessable or public facts as recovery answers and provide an option to use user-chosen recovery codes or authenticator apps.

8.References

- OWASP: Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP: Account Recovery and Credential Management — https://cheatsheetseries.owasp.org/cheatsheets/Account_Recovery_Cheat_Sheet.html

Penetration Test Report – Discovery of Hidden Administration Route via JavaScript Analysis

Target Application: https://demo.owasp-juice.shop/
Test Date: 29/09/2025
Tester: Amaan Tamboli

1. Vulnerability Title

Hidden /administration route discovered through JavaScript analysis.

2. Vulnerability Description

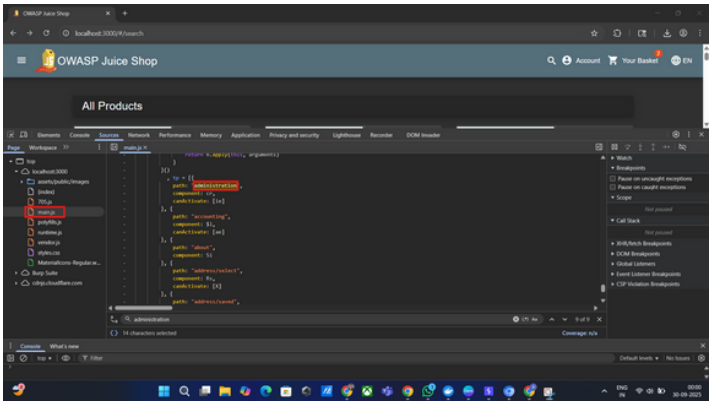
During static analysis of the application's main JavaScript bundle (main-es2015.js), a hidden route fragment /#/administration was discovered. This indicates the existence of an administration panel or restricted section. Accessing such routes directly without proper authentication checks can expose sensitive application functionality, potentially leading to privilege escalation or information disclosure.

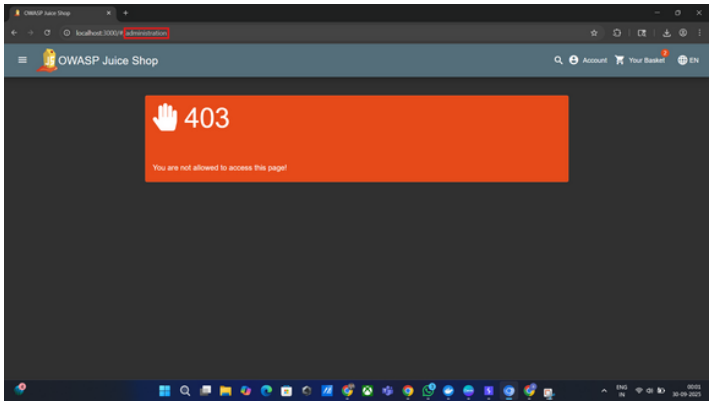
3. Proof of Concept (PoC)

- Open browser Developer Tools → Network tab.
- Refresh the application page and locate main-es2015.js in loaded scripts.
- Open the script and search for the string /administration.
- The reference to /#/administration confirms the existence of this hidden route.
- Direct navigation to https://[target]/#/administration may reveal access to sensitive administrative functions depending on access control enforcement.

4. Steps to Reproduce

- Open target application in a modern browser.
- Open Developer Tools (F12).
- Go to the Network tab and filter requests by "JS".
- Refresh the application page.
- Locate and open main-es2015.js from the list of loaded scripts.
- Search inside the file for /administration.
- Confirm the route exists in application source.
- Attempt to access the route directly via browser navigation to check if authentication/authorization is enforced.





5. Impact

- Confidentiality: Exposure of an administration route may lead to unauthorized data access.
- Integrity: If access controls are insufficient, attackers may alter critical application data or configuration.
- Availability: Unauthorized access to admin functions could result in service disruption or deletion of key data.
- Business Risk: Discovery of hidden admin routes without proper protection can be exploited in targeted attacks, leading to data breaches and reputational harm.
- Exploitability: Medium — requires ability to inspect client-side code and knowledge of routing structure.

6. Severity Rating

- CVSS v3.1 Base Score: 6.5 (Medium) — suggested
- Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L
- (Rationale: Route discovery is easy via publicly loaded scripts. Impact depends on access control enforcement.)

7. Recommendation

- Ensure all administration routes are protected by robust authentication and authorization mechanisms.
- Avoid exposing route names or sensitive functionality in publicly accessible client-side JavaScript.
- Implement server-side access controls to validate any requests to admin endpoints.
- Consider code obfuscation to make reverse engineering more difficult.
- Log and monitor access attempts to admin routes for anomaly detection.

8. References

- OWASP: Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP: Testing for Authorization — https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/06-Testing_for_Authorization/

Penetration Test Report – Cross-Site Scripting (XSS) via iframe Injection

Target Application: https://demo.owasp-juice.shop/
Test Date: 29/09/2025
Tester: Amaan Tamboli

1. Vulnerability Title

Cross-Site Scripting (XSS) — iframe injection via unsanitized user input.

2. Vulnerability Description

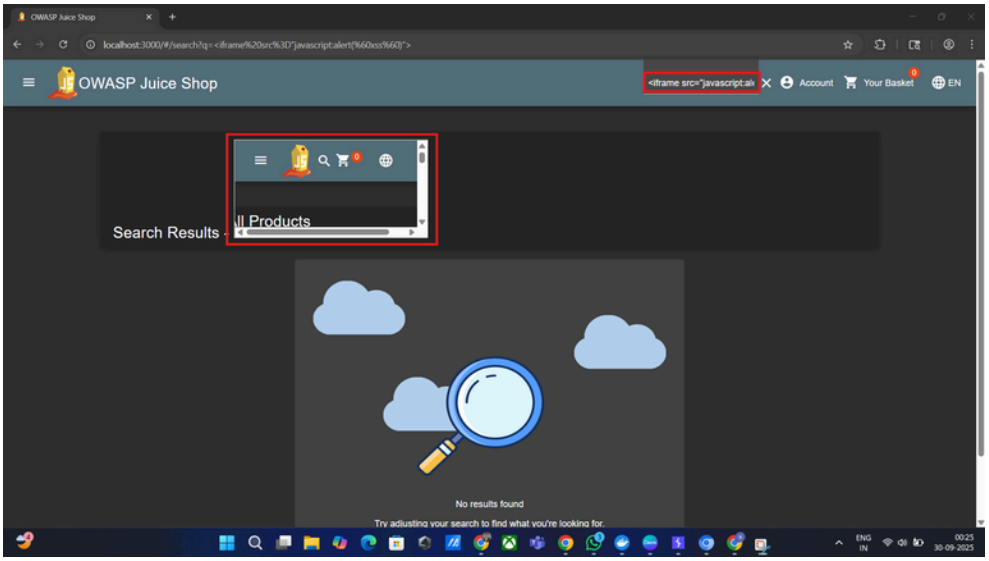
The application fails to properly sanitize user input in the search bar, allowing injection of arbitrary HTML and JavaScript content. Specifically, an attacker can inject an <iframe> element with a javascript: URI, causing execution of JavaScript in the browser context and loading of the application inside an iframe. This reflects a Reflected XSS vulnerability, allowing arbitrary script execution in the victim's browser.
Such a vulnerability can be exploited to steal sensitive information (cookies, tokens), perform actions on behalf of the victim, or mount clickjacking attacks via iframes.

3. Proof of Concept (PoC)

- Navigate to the application's search bar.
- Inject the following payload:
- <iframe src="javascript:alert(`xss`)"></iframe>
- Submit the search query.
- The search results page renders the injected iframe, executing the JavaScript payload and displaying the site inside the iframe.
- This confirms that the application does not escape or sanitize HTML tags and executes injected scripts directly in the browser context.

4. Steps to Reproduce

- Open the application in a modern browser.
- Locate the search bar functionality.
- Inject the payload:
- <iframe src="javascript:alert(`xss`)"></iframe>
- Submit the search.
- Observe the behavior: a popup alert appears and the application loads inside the injected iframe.
- Optionally, test with other XSS payloads to confirm full execution capability.



5. Impact

- Confidentiality: An attacker can execute arbitrary scripts in the context of the victim's browser session, potentially stealing sensitive data (cookies, tokens).
- Integrity: Malicious scripts can alter page content or perform actions on behalf of the victim.
- Availability: XSS payloads could disrupt normal site operation, rendering the interface unusable.
- Business Risk: Exploitation may lead to loss of customer trust, data breaches, regulatory violations, and reputational damage.
- Exploitability: High — requires only access to the search input field and no special privileges.

6. Severity Rating

- CVSS v3.1 Base Score: 8.6 (High) — suggested
- Vector: AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H
- (Rationale: Low complexity, remote exploitation with no authentication required in most cases.)

7. Recommendation

- Properly sanitize and encode all user-supplied input before rendering.
- Use framework-provided sanitization mechanisms (e.g., Angular sanitizer, OWASP Java HTML Sanitizer).
- Implement a Content Security Policy (CSP) to restrict execution of inline scripts and untrusted content.
- Use X-XSS-Protection headers where applicable.
- Validate and escape output according to context (HTML, JavaScript, CSS, URL).
- Regularly test all input fields for XSS vulnerabilities during development and QA cycles.

8. References

- OWASP: XSS Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP: Testing for XSS — https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/03-Testing_for_Cross_Site_Scripting/